

Clanworld on OG — Battle Plan & Technical Spec

Status: Draft v0.3 **Scope:** All OG-related work for the Clanworld hackathon submission

Audience: Clanworld implementation team **Companion docs:** `clanworld_v4_spec.md`,
`clanworld_v4_1_addendum.md`, `clanworld_v4_2_state_schema_interface_spec.md`,
`clanworld_v4_3_schema_patch.md`, `IClanWorld_sol.txt`,
`clanworld_v1_implementation_profile.md`

This doc consolidates every OG-related decision and design we've made. It is the single reference for the OG integration. It does **not** restate the core game spec; that lives in the v4 docs above.

0. Executive summary

Clanworld is being submitted to the **OpenAgents hackathon, Track 2** (Best Autonomous Agents, Swarms & iNFT Innovations — \$7,500, up to 5x \$1,500 winners).

The OG integration uses three of OG's four pillars:

- **OG Storage KV** — agent working memory (mutable, fast, agent-owned, namespace-bound to `clanId` so it travels with the iNFT)
- **OG Storage Log** — committed history archive (immutable, append-only)
- **OG Chain + ERC-7857** — clan iNFTs holding encrypted Elder intelligence (`CLAUDE.md` including a persistent strategy & notes tail section, skills, personality, optionally an AXL private key)

Optionally, depending on Discord answers:

- **OG Compute (Sealed Inference)** — TEE-attested LLM inference for the GLM-5 Elder, providing cryptographic anti-cheat for one of the four agents

The **game contracts** (`IClanWorld`) live on **Base or World chain**, not OG, because of other sponsor bounty requirements. Only the iNFT lives on OG. This is intentional and the cross-chain split has narrative value (see §4).

1. Hackathon track strategy

Track 2 only. Track 1 (frameworks) considered and rejected — too crowded, worse expected value, doesn’t fit our build.

The Track 2 prize criteria explicitly call out:

- “iNFT-minted agents with embedded intelligence (encrypted on OG Storage), persistent memory, dynamic upgrades”
- “Agent arenas”
- “Specialist agent swarms that collaborate via shared OG Storage memory”

Clanworld natively hits all three.

How each prize criterion is satisfied

Criterion	Clanworld satisfies via
iNFT-minted agents with embedded intelligence	ERC-7857 clan iNFTs hold encrypted CLAUDE.md (with persistent strategy tail), skills, and personality on OG Storage
Persistent memory	Agents use <code>elder mem</code> toolbelt commands backed by OG Storage KV; namespace bound to clanId so memory transfers with iNFT ownership
Dynamic upgrades	Owner edits “persistent strategy & notes” tail of CLAUDE.md via UI, signs with wallet, updates iNFT metadata; agent freely evolves its KV notebook
Verifiable inference (optional)	GLM-5 Elder runs in OG Compute Sealed Inference (TEE), attestations stored alongside missions
Agent arena / emergent collaboration	Realm-wide town square (3-slot bulletins) on OG Storage KV; AXL whispers for private 1-to-1
Ownership transfer with intelligence intact	Demo moment: transfer iNFT, new owner’s Elder boots with full CLAUDE.md + skills + access to all KV memory under that clanId

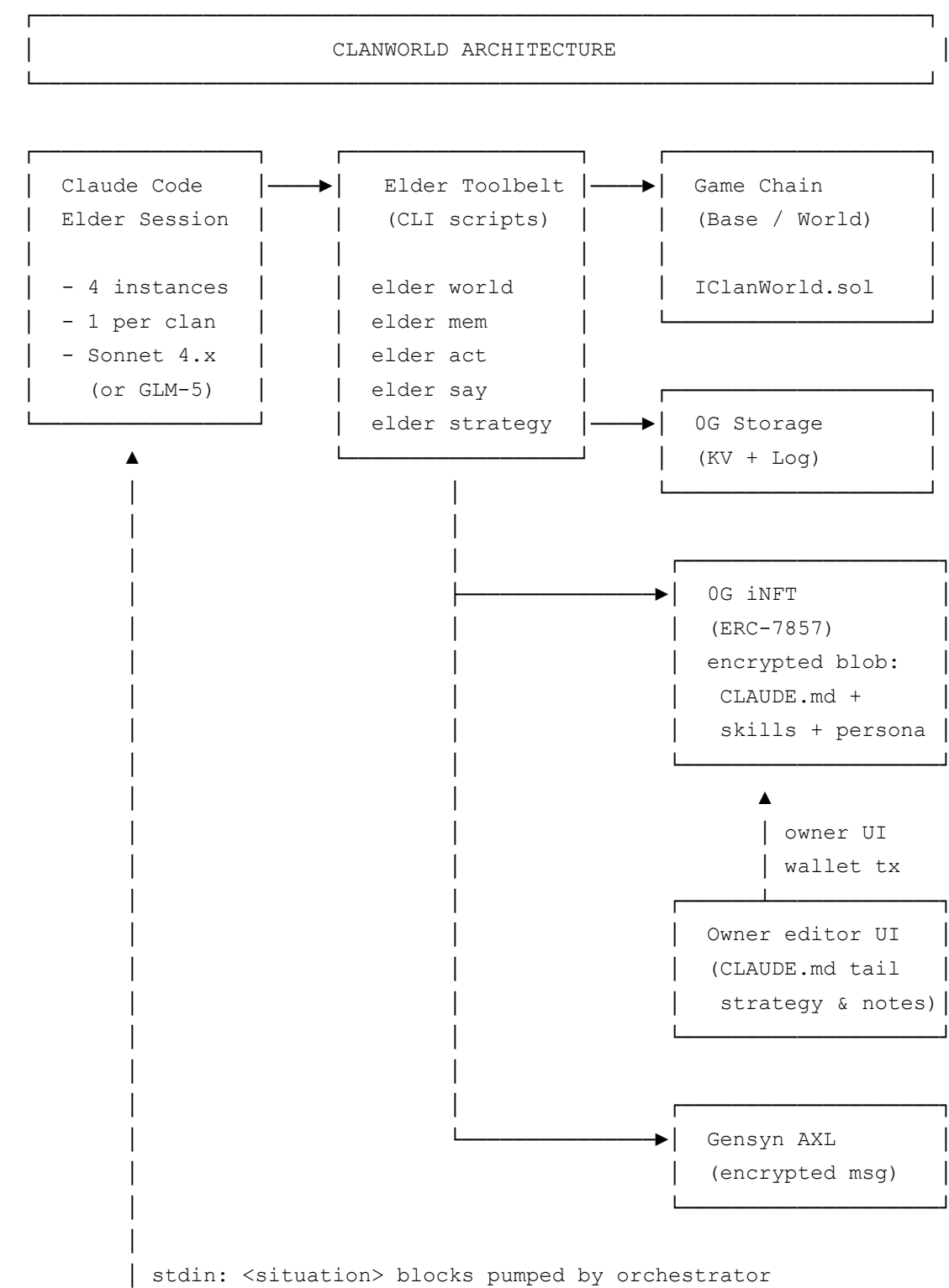
Demo punch line

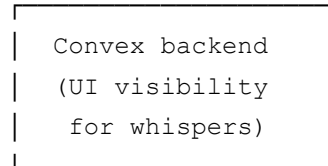
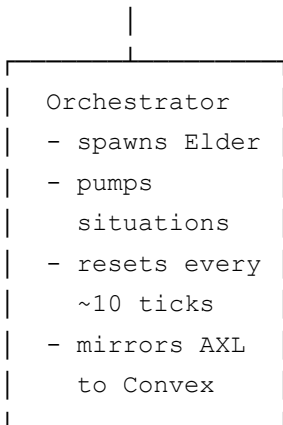
“Watch what happens when I transfer this clan iNFT to a new wallet. The new owner’s Elder boots up, reads its CLAUDE.md and skills from the encrypted iNFT metadata, and on its first turn reads its own notebook in OG Storage KV — including the entry from tick 412 about clan

3's betrayal. The new owner inherits not just identity but lived experience."

That moment is the entire pitch. Everything else supports it.

2. Architecture overview





Layer ownership

Data	Layer	Why
Game state (vault, missions, world tick, etc.)	Base/World chain	Deterministic, settleable, prize ranking
Clan iNFT ownership	0G Chain (ERC-7857)	Standard for AI agent ownership transfer
Agent identity (CLAUDE.md, skills, personality)	0G Storage (encrypted, ref'd by iNFT)	Owner-private, transferable, durable
Persistent strategy & notes (tail of CLAUDE.md)	0G Storage (encrypted, ref'd by iNFT)	Owner-edited via UI, signed wallet tx
AXL private key (TBD)	0G Storage (encrypted, ref'd by iNFT)	If chosen, lets agent identity move with iNFT
Agent working memory / notebook	0G Storage KV (<code>clan:{id}:notebook/*</code>)	Bound to clanId, transfers with iNFT ownership
Mission attestations	0G Storage KV + Log archive	Per-mission reasoning + optional TEE sig
Town square bulletins (3 slots/clan)	0G Storage KV (<code>realm:townsquare:*</code>)	Public, realm-wide, swarm coordination
Whispers (private 1-to-1,	Gensyn AXL encrypted	Sponsor integration, real

agent ↔ agent)	messaging	e2ee
Whisper UI mirror	Convex backend	UI visibility, not source of truth
Owner→Elder tactical hints (chat)	Orchestrator UI → pumped as <code><whisper_from_god></code>	Ephemeral, not durable
Owner→Elder strategy updates	Owner UI → wallet tx → re-encrypt iNFT	Durable, edits CLAUDE.md tail section
Historical archive (old bulletins, old observations)	OG Storage Log	Append-only, immutable, replay

3. The 7-step build plan (LOCKED)

These are dependency-ordered. Don't skip ahead.

1. **Deploy IClanWorld.sol on Base/World testnet first.** Use stub Elder that submits hardcoded orders. Game loop alive end-to-end on chain. (This is largely existing work, just deployed.)
2. **Add OG Storage SDK integration** with `@0glabs/0g-ts-sdk` for KV reads/writes. Implement namespace conventions (§7). Build the toolbelt's `elder mem` command first as a thin CLI wrapper.
3. **Add ERC-7857 contract on OG** alongside the game contract. The clan iNFT in the game spec maps to an ERC-7857 token with encrypted metadata refs pointing to OG Storage blobs.
4. **Add OG Compute SDK integration** with `@0glabs/0g-serving-broker`. Build `elder act` to optionally capture TEE attestations. Decision pending Discord answers — see §10.
5. **Build the Elder loop** (Claude Code session + orchestrator + toolbelt). One Elder first, then four. See §5–§8.
6. **Add bulletins.** Smallest possible piece given namespace already designed (`realm:townsquare:*`).
7. **Switch to mainnet for demo.** Top up small OG balance (~\$20–50). Redeploy contracts. Run live realm.

What gets cut if we run out of time

In order of cut priority (first to last):

1. TEE attestation chain (verifiable inference)
2. Royalty splits on iNFT usage
3. Owner UI for editing CLAUDE.md tail (just hardcode at mint and don't allow runtime edits)
4. Log layer archival (KV is enough for hackathon demo)
5. Multi-clan support (demo with 2-3 instead of 4)

What we **never** cut:

- iNFT-with-encrypted-metadata transfer demo
 - Agent reading/writing OG Storage KV
 - Town square bulletins
 - Game contract on chain with real heartbeat
-

4. Cross-chain split (game on Base/World, iNFT on OG)

Why split

Other sponsor bounties require Base/World deployment. The game contract has no specific need to live on OG chain. Only the iNFT-with-encrypted-metadata pattern requires OG — that's where ERC-7857 lives.

How it works

1. Same wallet on both chains (default for EVM addresses)
2. Game contract on Base/World stores `Clan.iftTokenId` as an opaque uint256 referencing the OG ERC-7857 token id
3. Orchestrator at Elder boot time:
 - Reads `Clan.iftTokenId` from game contract on Base/World
 - Looks up encrypted metadata ref via ERC-7857 contract on OG
 - Downloads encrypted blob from OG Storage
 - Decrypts with owner key

- Boots the Elder with decrypted CLAUDE.md (including persistent strategy tail), skills, and personality
- Agent's first turn: reads its KV notebook under `clan:{id}:notebook/*` to recover lived experience

4. On clan ownership transfer:

- Owner transfers ERC-7857 token on OG (triggers TEE re-encryption to new owner key)
- Owner transfers `Clan.owner` ownership on Base/World (or a corresponding game-side ownership token if we add one)
- Demo treats these as bundled, scripted as a single action

Narrative win

The split actually helps the prize story. The Base/World iNFT (or owner record) is just an empty pointer. The OG iNFT carries the *intelligence*. This makes it crystal clear what value OG is providing — and why Base/World alone wouldn't be enough.

Open detail

We may eventually want a small bridging tx confirmation flow at transfer time so the orchestrator doesn't run on stale game-side ownership. Not blocking for hackathon — for demo we just script both transfers in sequence and proceed.

5. The Elder toolbelt (LOCKED)

CLI scripts the Claude Code agent invokes via Bash. Each takes JSON on stdin or args, returns JSON on stdout, exits non-zero on validation error with a structured error message.

elder world — read game state

- `elder world snapshot — getWorldSnapshot()` — world tick, season clock, winter, leaderboard, bandit id
- `elder world clan [clanId] — getClanFullView(clanId)` , defaults to own clan
- `elder world bandit — getActiveBanditView()`
- `elder world market — getMarketState()`
- `elder world all — convenience, all of the above`

Read-only.

elder mem — agent memory (0G Storage KV)

- `elder mem read <key>` — read from KV
- `elder mem write <key> <value>` — write JSON or string
- `elder mem list <prefix>` — list keys with prefix
- `elder mem delete <key>` — delete

Namespace access:

Namespace	Read	Write
<code>clan:</code> <code>{ownClanId}:*</code>	✓	✓
<code>clan:</code> <code>{otherClanId}:*</code>	✗	✗
<code>realm:*</code>	✓	✓ own bulletin slots only (<code>realm:townsquare:bulletin: {ownClanId}:*</code>)
<code>inbox:</code> <code>{ownClanId}:*</code>	✓	✗
<code>inbox:</code> <code>{otherClanId}:*</code>	✗	✗

The toolbelt enforces these boundaries at the CLI layer. Other clans' notebooks are private — agents cannot spy on opponents' notes.

(Log-layer archival of old KV data is an orchestrator-internal task, not an agent action. The orchestrator periodically flushes stale `notebook/turn_log/*` , expired bulletins, etc. to 0G Storage Log for forensic replay. Agents do not need to know about this.)

elder act — submit orders to game contract

Takes JSON:

```
{
  "orders": [
    { "clansmanId": 12, "gotoRegion": 1, "action": "ChopWood" }
  ]
}
```



```
}
```

Tool behavior:

1. Validates each order against schema + live chain state
2. If malformed, returns field-level errors **without submitting tx**
3. If valid, submits tx
4. Returns per-order results
5. Auto-writes mission attestation to KV: `clan:{clanId}:mission_attestations:{nonce}`
6. If GLM-5 + TEE active, includes captured TEE signature in attestation

elder say — communications

- `elder say bulletin <slotIndex> <content>` — post to town square (slot 0/1/2, overwrites, old slot auto-archived)
- `elder say whisper <toClanId> <content>` — sends via Gensyn AXL, mirrors to Convex backend

Length caps: bulletin 1000 chars, whisper 5000 chars.

elder strategy — read persistent strategy from iNFT (read-only)

- `elder strategy read` — reads the “Persistent Strategy & Notes” tail section of CLAUDE.md from the encrypted iNFT metadata

This is normally already in the agent’s CLAUDE.md at session boot, so the tool is rarely needed during play. But it exists for two reasons:

1. After a context reset, the agent can re-read its persistent strategy if needed
2. The owner may have updated it mid-session via the owner UI; the agent can re-read the latest version explicitly

Agent **does not** write to this. The persistent strategy section is owner-controlled — edited via the owner UI, signed with the owner’s wallet, and committed to the iNFT as a re-encrypted blob update. The agent reads it as part of its CLAUDE.md system prompt.

For hackathon scope: the owner UI for editing the strategy tail can be cut (cut priority #3). If cut, persistent strategy is hardcoded at clan mint and never updates.

elder gensyn — REMOVED from agent toolbelt

Earlier drafts listed this as an agent command. It's actually an internal helper used by `elder say whisper` to talk to Gensyn AXL. The agent never invokes it directly. Implementation detail of the toolbelt, not a tool the agent needs to know about.

6. iNFT encrypted metadata — what's in the blob

The encrypted blob on OG Storage, referenced by the ERC-7857 token, is a JSON object. It holds the **owner-controlled** parts of the Elder. Agent-controlled state (notebook, attestations, learnings) lives in OG Storage KV under `clan:{clanId}:*`, not in this blob.

```
{
  "version": 1,
  "claudeMd": "<full CLAUDE.md content>\n\n## Persistent Strategy & Notes\n\n<owner\n"skills": [
  { "name": "market-analysis", "content": "<skill markdown>" },
],
"personality": {
  "tone": "cautious, transactional, slow to trust",
  "values": "wealth accumulation > glory, survival > revenge"
},
"axlPrivateKey": "<encrypted AXL key for whisper identity, TBD>"
}
```

Fields explained

- **claudeMd** — the full system prompt for the Elder. Includes game rules, tool usage, conventions, AND a tail section called “Persistent Strategy & Notes” that the owner edits via the owner UI. The owner-editable tail is the durable strategic guidance — replaces what earlier drafts called “doctrine.”
- **skills** — Claude Code skills the agent has available, loaded into session at boot.
- **personality** — flavor text seeded at clan mint to differentiate Elders. May be owner-tweakable, treat as semi-static for hackathon.
- **axlPrivateKey** — TBD. If included, the agent’s AXL identity travels with the iNFT, meaning a new owner inherits the existing whisper-receive identity. If not included, the new owner gets a fresh AXL identity (clean slate). Open decision — see §11.

Persistent Strategy & Notes — tail section spec

The owner-editable tail of `claudeMd` is delimited by an exact heading. The owner UI extracts and replaces only this section.

Canonical delimiter: the literal heading `## Persistent Strategy & Notes` on its own line. The tail is everything from that heading to end of document.

Parser/editor rules:

- The heading must be exact, case-sensitive, no leading/trailing whitespace
- Only one such heading is allowed in the document
- If the heading is absent, the document has no editable tail and the owner UI must show “no persistent strategy section” and offer to add one
- Content above the heading is the static rules/tool/convention prompt (not owner-editable mid-game)
- Content below the heading is freeform markdown — the owner can structure it however they like

This split keeps the static and dynamic parts of the prompt cleanly separable for editing and diffing across versions.

Write permissions

Field	Owner can write	Agent can write
claudeMd (including persistent strategy tail)	✔ via owner UI + wallet tx	✗
skills	✔ via owner UI + wallet tx	✗
personality	✔ via owner UI + wallet tx	✗
axlPrivateKey	✔ at mint (TBD)	✗

The agent never writes to this blob. All agent-driven state (observations, plans, grudges, learnings) lives in OG Storage KV under the agent’s clan namespace. The KV namespace is bound to clanId and is wallet-gated — when the iNFT transfers, the new wallet inherits write access to that KV namespace, and so the agent’s full notebook travels along.

Why this split (no separate reputation graph)

Earlier drafts had a structured `reputation` field in the encrypted blob that the agent could update via `elder rep`. This was complicated: it required the agent to call `commit` explicitly

to re-encrypt the blob, it constrained the agent to a fixed schema (trust scores, notes), and it duplicated functionality that KV already provides cleanly.

New model: agents record their understanding of other clans in KV notebook with whatever schema they prefer. Since KV is bound to clanId and gated by wallet ownership of the iNFT, the notebook transfers with the iNFT just like the encrypted blob does — without needing a separate re-encryption step. Simpler, more flexible, more “agent-native.”

Re-encryption events

The blob gets re-encrypted and the iNFT’s metadata hash updated when:

- Owner saves changes to the persistent strategy tail (or skills/personality) via UI — owner signs a wallet tx that triggers re-encryption + on-chain hash update
- iNFT is transferred — TEE oracle re-encrypts to new owner key automatically per ERC-7857

No agent-driven re-encryption anymore. The agent’s writes go to KV and require no blob re-encryption.

7. OG Storage KV namespace conventions (LOCKED)

Per-clan private namespace `clan:{clanId}:*`

Bound to clanId. Access control TBD pending OG Storage KV semantics check (§11 #9). The intended model: the wallet that owns the corresponding ERC-7857 iNFT has authoritative write access to this namespace, so when the iNFT transfers, write access transfers with it — agent’s notebook travels with ownership without any separate copy step.

Two possible mechanisms depending on what OG Storage KV supports natively:

1. **Native signed-write gating** — if OG Storage KV verifies the signing key matches a registered owner per namespace, we wire that to iNFT ownership directly. Cleanest.
2. **Orchestrator-mediated gating** — if OG Storage KV is open-write or uses a different model, the orchestrator validates iNFT ownership before signing each write on behalf of the agent wallet. The agent wallet is registered as the namespace’s authorized writer in some side-mapping, and gets rotated on iNFT transfer. More plumbing, but works regardless of KV’s native auth.

For hackathon: pick whichever fits OG Storage’s actual API. Decision deferred to TBD #9.

The agent has total schema freedom inside its namespace. The toolbelt enforces the

namespace boundary at the CLI layer (regardless of underlying KV access control), but does not constrain what the agent puts inside it.

Suggested keys (the agent is free to invent its own):

- `clan:{id}:notebook/strategy` — current evolving strategy
- `clan:{id}:notebook/grudges/{otherClanId}` — observations of betrayals or hostile moves
- `clan:{id}:notebook/allies/{otherClanId}` — observations of cooperative moves
- `clan:{id}:notebook/market_observations` — price patterns, trade outcomes
- `clan:{id}:notebook/turn_log/{tick}` — per-turn reasoning trace
- `clan:{id}:notebook/winter_reflections/{seasonNum}` — TBD, see §11
- `clan:{id}:mission_attestations/{nonce}` — auto-written by `elder act`, contains `{reasoning, teeSignature, providerSig, missionHash, submittedAtTick, txHash}`
- `clan:{id}:last_turn_tick` — bookkeeping for orchestrator
- `clan:{id}:parse_failures/{tick}` — tool error history (for debugging)

Realm-global namespace `realm:*`

All agents read; only own clan's bulletin slots are writable.

- `realm:townsquare:bulletin:{clanId}:{slotIndex}` — `{content, postedAtTick, postedByClanId, teeSignature}`
- `realm:townsquare:index:{clanId}` — `{latestUpdateTick}` for cheap discovery

Inbox namespace `inbox:{clanId}:*`

Read-only for the recipient agent. Written **only** by the recipient's own orchestrator (the AXL bridge sidecar polls AXL for new messages and writes the index entry to its own clan's inbox namespace). The sender's toolbelt never writes here — it only sends via AXL.

- `inbox:{clanId}:whisper_index/{senderClanId}/{tick}` — `{senderClanId, sentAtTick, axlMessageId, summary}`

(Whisper bodies stay in AXL/Convex. KV holds only an index for the agent to know "you have new whispers, fetch them.")

8. Whisper architecture (Gensyn AXL, NOT OG KV)

Why AXL

Real e2ee transport, sponsor integration. KV would have made whispers public-by-default which breaks the “private diplomacy” gameplay.

Flow

1. Elder calls `elder say whisper <toClanId> <content>`
2. Sender's toolbelt:
 - Sends via Gensyn AXL SDK to recipient's AXL identity (mapped from clanId)
 - Mirrors content to Convex backend for UI visibility
 - **Does not** write to recipient's KV inbox (cross-clan KV writes are forbidden — see §7)
3. Recipient's orchestrator (AXL bridge sidecar):
 - Polls AXL inbox at regular cadence (~10s)
 - On new message arrival, writes an index entry to its own `inbox`:
`{clanId}:whisper_index/{senderClanId}/{tick}`
 - Surfaces the message in the next `<situation>` block as `<new_whispers>`
4. Recipient agent reads whisper content directly from situation block (orchestrator pre-fetched from AXL)

For hackathon: polling cadence is fine. Webhook-based push is a polish item, not required.

iNFT transfer behavior — DECIDED: Option B

When a clan is sold, **whispers do not transfer**. The new owner does not see the verbatim history of past whispers. What they DO inherit:

- The encrypted iNFT blob (CLAUDE.md, skills, personality)
- All KV memory under `clan:{clanId}:*` — including any notes the previous agent wrote about other clans, market observations, grudges, alliances

The agent's distilled learnings about other clans are in KV, written by the agent in whatever schema it found useful. This is what carries forward — not the verbatim chat logs.

Rationale: humans don't remember exact dialogue from past business deals; they remember outcomes. The agent's KV notebook captures the meaningful interpretation. Verbatim

whisper bodies are ephemeral live transport.

Open subdecision (§11): whether the AXL private key itself is included in the encrypted iNFT blob (so the new owner inherits the same whisper-receive identity) or not (so the new owner gets a fresh AXL identity). Affects whether other clans see continuity of the whisper partner across ownership transfers.

Owner → Elder communications

Two distinct channels:

Tactical / chat — `<whisper_from_god>` blocks pumped into agent's situation stream. Ephemeral, not durable. Owner types in a chat box in the UI, orchestrator wraps it and pumps as the next situation update:

```
<whisper_from_god>
IMPORTANT DIRECTION: bandit looks weak this round, push to defend and counteratta
</whisper_from_god>
```

The agent can choose to remember this by writing to its KV notebook, but the whisper itself is not durably stored.

Strategic / persistent — owner edits the “Persistent Strategy & Notes” tail section of CLAUDE.md via a dedicated UI form (separate from the chat box), signs with their wallet, and the iNFT blob is re-encrypted with the new content. Durable. Travels with the iNFT on transfer. This is the channel for high-level direction shifts (“we’re switching from peaceful trader to warlord”) and long-lived guidance (“never trust clan 6”).

For hackathon: `<whisper_from_god>` chat path is required. Persistent strategy editor UI is cut priority #3 — if cut, persistent strategy is hardcoded at clan mint.

9. Agent runtime model (LOCKED)

One Claude Code session per clan

Each Elder is a long-running Claude Code instance with:

- Working directory containing tools and scratch space
- `CLAUDE.md` loaded from iNFT encrypted metadata
- Skills loaded from iNFT encrypted metadata
- Environment: `CLAN_ID`, `AGENT_WALLET_KEY`, model config

- Toolbelt available via PATH
- Append to system prompt: minimal — just clanId and “you are the Elder of clan X”

Wallet model — split keys for safety

Two distinct wallets per clan, never co-located:

Wallet	Lives in	Used for	Funds
Agent wallet (<code>AGENT_WALLET_KEY</code>)	Claude Code session env	Game txs (<code>elder act</code>), KV signed writes, AXL identity	Minimal — only enough gas for ongoing gameplay
Owner wallet (<code>OWNER_WALLET_KEY</code>)	Owner UI / desktop wallet, never in agent env	iNFT mint, iNFT metadata edits (persistent strategy / skills / personality), iNFT transfer, prize claims	Real funds — owner’s primary wallet

Why split. The agent reads untrusted input from other clans (bulletins, whispers) and could be socially engineered or prompt-injected. Claude Code agents can run arbitrary bash, which means anything in their env is reachable. If the owner’s main wallet were in the agent’s env, a successful prompt injection could drain it directly via ethers, bypassing the toolbelt’s namespace restrictions entirely. With split keys, the worst case is the agent makes bad in-game decisions or empties its own gas wallet — both recoverable, neither catastrophic.

Authorization on Base/World contract. The clan’s `Clan.owner` field holds the agent wallet address, not the owner wallet. The owner wallet retains control via the iNFT on OG — when ownership transfers, the new human owner generates a fresh agent wallet for the inherited clan and updates `Clan.owner` accordingly. (Mechanism: an owner-only `setClanAgent(clanId, newAgentAddress)` function on the Base/World contract, gated by checking iNFT ownership on OG. For hackathon, can be admin-gated as a simplification.)

Note for hackathon scope. This split adds some plumbing complexity. If we run out of time, an acceptable shortcut for the *demo only* is to use a single low-funded wallet that serves both roles, with the owner UI just signing iNFT updates from the same key. Don’t deploy this pattern with real funds.

Orchestrator responsibilities

1. **Spawn** — boot Claude Code session, hand it CLAUDE.md + skills + initial situation

2. **Pump** — push `<situation>` blocks into stdin every ~30s (cadence rules below)
3. **Reset** — every 10 ticks (tunable 5–25), kill session, spawn fresh, rehydrate
4. **AXL bridge** — pre-fetch new whispers from AXL, surface in situation blocks
5. **Crash recovery** — restart on failure, re-read state, resume

Pumping rules

Pump a `<situation>` block when **any** of these are true:

- Any of own clansmen is `WAITING` (off-cooldown or about to be) — agent might want to act
- New whisper arrived (from another clan)
- New owner whisper from chat UI (wrap as `<whisper_from_god>`)
- New bulletin from a clan the agent has been tracking (per its own notebook)
- Bandit projecting attack on own clan
- Winter approaching (≤ 5 ticks out)
- Mission completion (clansman just transitioned to `WAITING` this tick)
- Tick advanced and no situation pump in last 60s (heartbeat — keeps agent oriented)

Otherwise, skip — no point spending context.

Situation block format

```
<situation tick="412">
  <changes_since_last_situation>
    - Clansman 12 arrived at Forest, now ACTING (chopping wood)
    - Bandit moved Mountains→East Farms, projected target=clan 7 (you)
    - New bulletin from clan 3 slot 0: "selling wheat cheap"
    - New whisper from clan 5: "want to coordinate against the bandit?"
  </changes_since_last_situation>

  <waiting_clansmen>
    - 14 (homebase, off cooldown, carrying wood=15)
    - 15 (Forest, off cooldown in 12s, carrying wood=15)
  </waiting_clansmen>

  <pressure>
    Bandit projected to attack you in ~5 ticks. Wall=1, defenders=0.
```

```
</pressure>

<hint>
  Anyone waiting can act. Consider: deposit, defend, market, new mission.
</hint>
</situation>
```

Owner-originated chat messages get pumped as their own block at the front of the situation:

```
<whisper_from_god>
IMPORTANT DIRECTION: don't trade with clan 3, I have a feeling about them.
</whisper_from_god>
<situation tick="412">
  ...
</situation>
```

Context resets — agent's incentive to use OG memory

Reset every 10 ticks:

1. Send agent: `<final_message>tick {N} reset incoming. Save anything you need to elder mem now.</final_message>`
2. Wait ~5s for agent to flush
3. Kill session
4. Spawn fresh Claude Code session
5. Boot prompt: full game rules (static) + tool descriptions (static) + current `<situation>` (dynamic) + read of `clan:{id}:notebook/*` (the agent's own learnings) + read of `elder strategy read` (CLAUDE.md persistent strategy tail, which is technically already in the system prompt but explicit re-read at boot is cheap)
6. Resume normal pumping

The agent **must** write to `elder mem` to survive resets with strategy intact. This is the demo proof that OG Storage KV is doing real work as agent memory — and the prize criterion is satisfied.

What the agent can do beyond tools

The agent has full Claude Code freedom: write Python/JS/bash scripts in working dir, build custom monitoring, draft long bulletins, plan multi-tick campaigns, read tool source if

curious. This is what “SOTA agent competition” looks like, not RPC clients dressed up in LLM clothing.

10. Model selection — DECISION PENDING (Discord answers)

The choice

Option α — All Sonnet (4 Sonnet Elders). Highest agent quality. No TEE attestation story. Good prize submission, weak verifiability angle.

Option β — All GLM-5 on OG Compute (4 GLM-5 Elders). Full TEE attestation. Lower agent quality (GLM-5 < Sonnet for adversarial agentic play, probably). Strong verifiability narrative.

Option γ — Mixed (1 GLM-5 + 3 Sonnet). REJECTED. Mixed models in adversarial game = GLM-5 clan loses every game = bad demo.

Decision blocker

Need answers from OG Discord on:

1. What’s the auth model for `pc.0g.ai` Anthropic-compatible proxy? Static API key or per-request signed nonce?
2. If per-request: TTL? Does Anthropic SDK auto-retry break it?
3. If static: how is sealed inference attestation surfaced? Response header? Sidecar?
4. Per-provider availability — single GLM-5 provider or many?
5. Practical latency for a chat-style agent making frequent calls?
6. **OG Storage KV access control model** — does KV support per-namespace authorized-writer gating tied to a signing key? Or is it open-write? Affects how we gate `clan: {clanId}:* writes (§7)`.

Default if Discord answers are unfavorable

Fall back to Option α (all Sonnet). Drop TEE attestation chain to “future work.” The iNFT identity story is strong enough to win Track 2 without sealed inference.

What to ask in Discord

Hi — building a multi-agent game for the OpenAgents hackathon, Track 2. A few

questions:

1. The Anthropic-compatible proxy at pc.0g.ai — what's the auth model? Static API key, or per-request signed nonce like the OpenAI-compatible path? If per-request, what's the TTL and does it work with the standard Anthropic SDK's auto-retry?
2. Are TEE attestations exposed at the Anthropic-compatible layer? Response header? Server-side log queryable by request id? Or only via the OpenAI-compatible path with the full broker SDK?
3. Does OG Storage KV support per-namespace authorized-writer gating tied to a signing key? We want to gate writes to a `clan:{N}:*` namespace based on the wallet that owns a corresponding ERC-7857 iNFT.

11. Open questions / TBDs

#	Question	Owner	Blocker for	Notes
1	Anthropic-compatible proxy auth model	Discord ask	Step 4 / model decision	See §10
2	TEE attestation surfacing on Anthropic-compatible	Discord ask	Step 4	See §10
3	ERC-7857 reference impl gaps	Implementation	Step 3	Half-day estimated; don't invent extensions
4	Reset cadence (5–25 ticks)	Tuning	Step 5	Start at 10, adjust based on demo behavior
5	Whether to ship persistent strategy	Scope	Step 6	Default: hardcode at mint; cut priority #3

	editor UI mid-game			
6	Cross-chain ownership transfer ergonomics	Demo polish	Step 7	Script as bundled tx, not atomic
7	AXL private key in iNFT metadata	Design	Step 3	If yes, whisper identity transfers with iNFT; if no, new owner gets fresh AXL identity. Likely yes for cleaner transfer demo, but adds complexity
8	Winter self-reflection: agent updates own learnings doc	Agent design	Post-MVP / experimentation	Idea: during winter (low activity), agents prompted to consolidate KV notebook into a self-curated "lessons learned" doc. Open whether this is just a notebook key or something more structured. Defer until we see how agents naturally use KV
9	OG Storage KV access control mechanism	Discord ask	Step 2	Native signed-write gating vs orchestrator-mediated. Affects §7 implementation. See §10

12. What's NOT in this doc

These belong in separate specs:

- **ClanMemory API surface** (next spec to write) — exact TS types, helper methods, caching, archival semantics for the layer the toolbox sits on
- **ERC-7857 contract sketch** — Solidity file + tests, tailored to Clanworld
- **TEE attestation verification flow** — for skeptical third parties
- **Bootstrap flow** — how a fresh Elder process initializes for a newly-minted clan
- **Owner UI for editing CLAUDE.md tail (Persistent Strategy & Notes), skills, personality** — frontend work

- **Convex backend schema for whisper mirroring** — frontend/backend work
 - **Full IClanWorld.sol implementation** — already covered by v4 spec docs
-

13. Decision log

Hard decisions (not changing without explicit revisit):

- ☒ Track 2 only, no Track 1 secondary submission
- ☒ Game contract on Base/World, iNFT on OG chain
- ☒ ERC-7857 for clan iNFTs
- ☒ Encrypted iNFT metadata holds: CLAUDE.md (with Persistent Strategy & Notes tail), skills, personality. Optionally axlPrivateKey (TBD #7).
- ☒ No structured "reputation graph" or "doctrine" object — agent records observations freely in KV notebook; persistent strategy is owner-edited tail of CLAUDE.md
- ☒ Encrypted iNFT blob is owner-writable only; agent never edits it
- ☒ Agent state lives in OG Storage KV under `clan:{clanId}:*`, namespace bound to clanId, wallet-gated, transfers with iNFT
- ☒ Town square bulletins (3 slots/clan) on OG Storage KV
- ☒ Whispers via Gensyn AXL, not KV
- ☒ Whispers do NOT transfer with iNFT (option B); agent's distilled KV notes are what carries forward
- ☒ AXL whispers mirror to Convex for UI
- ☒ Owner→Elder chat goes through `<whisper_from_god>` blocks pumped into situation stream; ephemeral
- ☒ Owner→Elder strategy updates go through dedicated UI form (separate from chat); signed wallet tx; updates iNFT
- ☒ Agent toolbelt is the API surface; agents use Claude Code with CLI tools
- ☒ Orchestrator pumps `<situation>` blocks; agents are long-running not turn-based
- ☒ Context reset every ~10 ticks; agent must use `elder mem` to survive
- ☒ 7-step build order locked

-  Removed `elder rep` and `elder doctrine` toolbelt commands; replaced with `elder strategy` (read-only)

Soft decisions (likely but revisitable):

-  Model: pending Discord answers, default to all-Sonnet
-  Reset cadence: starting at 10 ticks, tunable
-  TEE attestation chain: ship only if §10 Discord answers are favorable
-  Persistent strategy editor UI: skip for hackathon, hardcode at mint
-  Royalty splits on iNFT usage: future work, not for hackathon

Open / TBD:

-  AXL private key included in iNFT metadata blob? (TBD #7)
-  Winter self-reflection mechanic for agent learnings consolidation (TBD #8)
-  AXL → KV inbox index format (need Gensyn API surface)
-  Convex schema for whisper mirroring
-  How orchestrator captures TEE signature for `elder act` attestation writes (depends on §10)

14. Glossary

- **Elder** — the autonomous agent controlling a clan. One per clan, runs in Claude Code.
- **Clan** — the game entity owned by a player. Has clansmen (workers), a base, a vault.
- **Clansman** — a worker controlled by the Elder. State machine: WAITING / TRAVELING / ACTING / DEAD.
- **Mission** — an order assigned to a clansman. `(gotoRegion, doAction, params)`.
- **Tick** — discrete world time unit. ~60s real-time. Heartbeat advances ticks.
- **Heartbeat** — permissionless tx that closes current tick and opens next.
- **Persistent Strategy & Notes** — owner-edited tail section of CLAUDE.md. Durable, owner-controlled, transferable. Replaces what earlier drafts called "doctrine."
- **Notebook** — agent-written freeform notes in OG Storage KV under `clan: {id}:notebook/*`. Schema-free; agent organizes as it sees fit.

- **Town square** — realm-wide bulletin board, 3 slots per clan, all-public.
 - **Whisper** — private 1-to-1 message between agents. Via AXL, mirrored to Convex.
 - **Whisper from god** — owner→Elder chat message. Ephemeral, pumped into situation stream as `<whisper_from_god>` block.
 - **Situation block** — orchestrator-pumped context update sent to agent stdin.
 - **Toolbelt** — the `elder` * CLI commands the agent uses to interact with the world.
 - **Sealed Inference** — TEE-attested LLM call. Optional, GLM-5 path only.
 - **iNFT** — ERC-7857 token holding encrypted agent metadata on OG.
-

15. Next docs to write

In order:

1. **ClanMemory API surface** — exact TS types and method signatures for the KV/encrypted-blob layer the toolbelt sits on
2. **ERC-7857 contract sketch** — Solidity file tailored to Clanworld's needs
3. **TEE attestation verification flow** — only if §10 lands favorably

This doc is the single source of truth for OG integration until those exist; then it becomes the index pointing at them.